

- *.libraries.yml* defines your libraries (mostly your JS, CSS files).
- *.breakpoints.yml* defines the points to fit different screen devices.
- *.theme* is the PHP file that stores conditional logic and data preprocessing of the variables before they are merged with markup inside the *.html.twig* file.
- */includes* is where third-party libraries (like Bootstrap, Foundation, Font Awesome, etc) are put. It is a standard convention to store them in this folder.

The basic requirement to create a new Drupal theme is to have Drupal localhost installed on your system.

Drupal 8 theme structure

A custom theme can be made by following the steps mentioned below.

Step 1: Creating the custom themes folder

Go to the Drupal folder in which you can find a folder named *Theme*.

- Enter the folder 'theme'.
- Create a folder 'custom'.
- Enter the folder 'custom'.
- Create a folder 'osfy'.

Start creating your theme files over here. The theme name taken here is *osfy*.

Step 2: Creating a YML file

To inform the website about the existence of this theme, we use *.yml* files. The basic details required in the YML are mentioned below:

1. Name
2. Description
3. Type
4. Core

```
name: osfy
description: My first responsive custom theme.
type: theme
package: custom
base theme: classy
core: 8.x
```

```
regions:
head: head
  header: header
  content: content
  sidebar: sidebar
  footer: Footer
```

```
Stylesheets-remove:
-"Remove Stylesheets"
```

We can proceed once the theme appears in the uninstalled section of your website's *Appearance* tab.

Open the Drupal website and check for the new theme in the *Appearance* section. It will be under the uninstalled list of themes in the *Appearance* tab.

Note: 1. *Base theme* indicates which base theme your custom theme is going to inherit. The default base theme provided by Drupal is 'Stable'.

2. *Regions* defines the regions in which your blocks are to be placed in your theme. If not declared, Drupal uses default regions from the core.

Step 3: Adding the .libraries.yml file:

We have indicated all the libraries comprising JavaScript and CSS styling, and now we will define them in the *libraries.yml* file.

```
global-components:
  version: 1.x
css:
  theme:
    css/style.css: {}
    includes/bootstrap/css/bootstrap.css: {}
```

We will use *style.css* for the theme styling and *bootstrap.css* for responsive display using Bootstrap libraries. *Style.css* resides in the *core/css* folder, whereas *bootstrap.css* resides in the *includes/bootstrap/css* folder.

Step 4: Creating theme regions

To better understand how Twig has made things easier, use the following code:

```
<?php print render($title_prefix); ?>
<?php if ($title): ?>
  <h1 class="title" id="page-title">
    <?php print $title; ?>
  </h1>
<?php endif; ?>

{{ title_prefix }}
{% if title %}
  <h1 class="title" id="page-title">
    {{ title }}
  </h1>
{% endif %}
```

The template file functions are:

html.html.twig - Theme implementation for the basic structure of a single page

page.html.twig - Theme implementation to display a single page

node.html.twig - Default theme implementation to display a node